

Java Backend Architecture

Interview Series

Chapter 17.8

Helm & Package Management

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

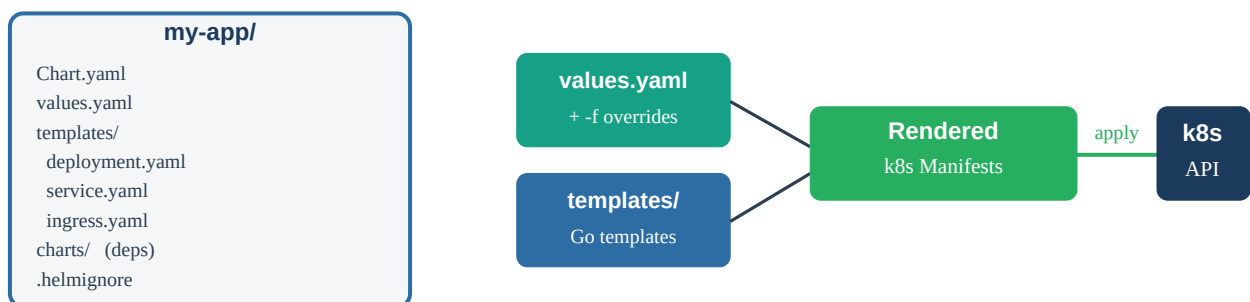
Chapter 17.8 — Helm & Package Management

Overview

Helm is the de-facto package manager for Kubernetes. Charts bundle templates, default values, and dependencies. Helmfile declaratively manages multi-chart environments. Kustomize provides overlay-based customization built into kubectl.

Chart Structure

Helm Chart Structure & Go Template Rendering



`{{ .Values.image.tag }}` resolved at render time | `helm template` to preview

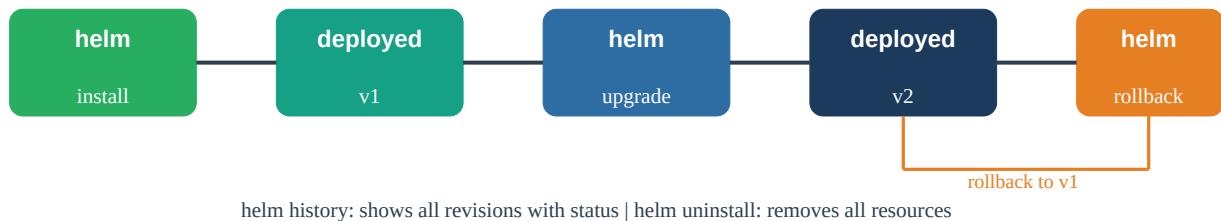
Chart.yaml	Metadata: name, version (chart), appVersion (app), description, dependencies
values.yaml	Default values referenced in templates via <code>{{ .Values.key }}</code>
templates/	Go template files rendered with values to produce k8s manifests
templates/NOTES.txt	Displayed after helm install/upgrade — usage instructions
templates/_helpers.tpl	Named template partials (<code>{{ define "app.name" }}</code>) reused across templates
charts/	Packaged chart dependencies (downloaded by helm dependency update)
Chart.lock	Exact versions of resolved dependencies — committed to Git

```

# Chart.yaml apiVersion: v2 name: my-app version: 1.3.0 # chart version (semver) appVersion:
"2.1.0" # application version dependencies: - name: postgresql version: "12.x.x" repository:
https://charts.bitnami.com/bitnami condition: postgresql.enabled # values.yaml image:
repository: my-registry/my-app tag: latest pullPolicy: IfNotPresent replicaCount: 2 service:
port: 8080 postgresql: enabled: true auth: database: mydb # templates/deployment.yaml (Go
template) apiVersion: apps/v1 kind: Deployment metadata: name: {{ include "my-app.fullname" .
}} labels: {{- include "my-app.labels" . | nindent 4 }} spec: replicas: {{ .Values.replicaCount
}} template: spec: containers: - name: {{ .Chart.Name }} image: "{{ .Values.image.repository
}}:{{ .Values.image.tag }}" ports: - containerPort: {{ .Values.service.port }}
  
```

Core Helm Commands

Helm Release Lifecycle



```
# Install a chart helm install my-release ./my-app -f prod-values.yaml --namespace prod
--create-namespace # Override specific values helm upgrade my-release ./my-app --set
image.tag=v2.1.0 --set replicaCount=5 # Multiple value files (later files override earlier)
helm upgrade my-release ./my-app -f base-values.yaml -f prod-values.yaml # Preview rendered
manifests (no install) helm template my-release ./my-app -f prod-values.yaml # Check release
history helm history my-release -n prod # Rollback to revision 2 helm rollback my-release 2 -n
prod # Uninstall (keeps history with --keep-history) helm uninstall my-release -n prod #
Dry-run (server-side validation) helm upgrade --dry-run --debug my-release ./my-app # Package
chart to .tgz helm package ./my-app --version 1.3.0 # Add OCI registry helm registry login
registry.example.com helm push my-app-1.3.0.tgz oci://registry.example.com/charts
```

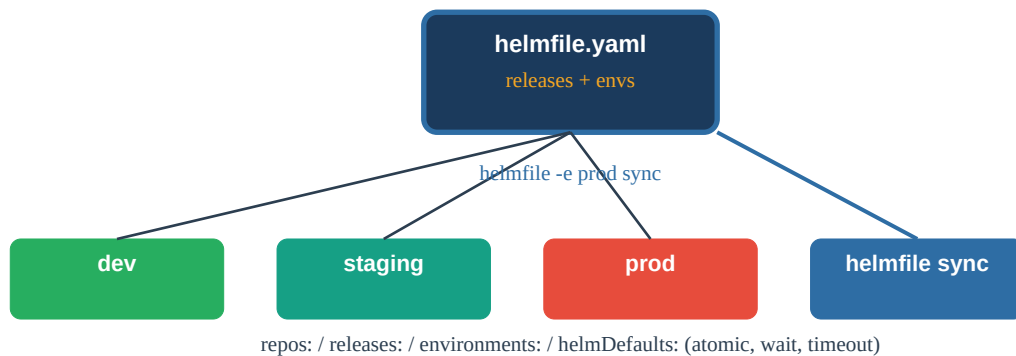
Helm Hooks

- **pre-install**: runs before any release resources created — DB migration Job
- **post-install**: runs after all resources created — smoke test, notify
- **pre-upgrade**: runs before upgrade — backup, validation
- **post-upgrade**: runs after upgrade — integration test
- **pre-delete**: runs before deletion — cleanup, archive
- **hook-weight**: execution order (lower runs first)
- **hook-delete-policy**: before-hook-creation (delete old), hook-succeeded, hook-failed

```
# DB migration Job (pre-install hook) apiVersion: batch/v1 kind: Job metadata: name: {{
.Release.Name }}-db-migrate annotations: "helm.sh/hook": pre-install,pre-upgrade
"helm.sh/hook-weight": "-5" "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec: template: spec: restartPolicy: Never containers: - name: migrate image: "{{
.Values.image.repository }}"{{ .Values.image.tag }}" command: ["/migrate", "--up"] env: -
name: DB_URL valueFrom: secretKeyRef: name: db-secret key: url
```

Helmfile — Declarative Multi-Chart Management

Helmfile: Multi-Chart Multi-Environment Releases



```
# helmfile.yaml repositories: - name: bitnami url: https://charts.bitnami.com/bitnami
helmDefaults: wait: true timeout: 600 atomic: true environments: dev: values: -
environments/dev/values.yaml prod: values: - environments/prod/values.yaml releases: - name:
postgresql namespace: db chart: bitnami/postgresql version: "12.5.0" values: -
postgresql-values.yaml - name: my-app namespace: app chart: ./charts/my-app needs: -
db/postgresql values: - my-app-base.yaml - {{ .Environment.Name }}-overrides.yaml set: - name:
image.tag value: {{ requiredEnv "IMAGE_TAG" }} # Deploy to production helmfile -e prod sync
helmfile -e prod diff # preview changes helmfile -e prod destroy # remove all
```

Kustomize — Overlay-Based Customization

- **Built into kubectl:** kubectl apply -k ./overlays/prod
- **Base:** common k8s manifests (deployment, service, configmap)
- **Overlay:** patches applied to base for each environment
- **patches:** strategic merge patch or JSON patch
- **namePrefix/nameSuffix:** add prefix/suffix to all resource names
- **images:** override image tag without editing templates
- No templating engine — pure YAML overlays (simpler but less powerful than Helm)

```
# base/kustomization.yaml resources: - deployment.yaml - service.yaml #
overlays/prod/kustomization.yaml bases: - ../../base namePrefix: prod- replicas: - name: my-app
count: 5 images: - name: my-app newTag: v2.1.0 patches: - patch: |- - op: replace path:
/spec/template/spec/containers/0/resources/limits/memory value: 2Gi target: kind: Deployment
name: my-app # Apply kubectl apply -k overlays/prod/ kubectl diff -k overlays/prod/
```

OCI Chart Registries

- Helm 3.8+ supports OCI (Docker registry) for chart storage
- Push/pull charts like Docker images: helm push chart.tgz oci://registry/repo
- No need for separate chart repository (chartmuseum, GitHub Pages)
- ECR, ACR, GAR, Harbor, Docker Hub all support OCI Helm charts
- helm show values oci://registry/repo/chart:1.0.0

Chart Testing (ct)

- ct lint: validates chart structure, Chart.yaml, values schema
- ct install: installs chart in temp namespace, runs helm test, cleans up
- Run in CI/CD pipeline before merging chart changes

- values_files in ci/ directory provide test-specific values
- chart-testing tool from helm/chart-testing GitHub repo

Interview Q&A; — Helm & Package Management

Q1. What is Helm and why is it used?

Helm is a package manager for Kubernetes that bundles related k8s manifests (Deployment, Service, ConfigMap, Ingress) into a Chart. It provides: templating (Go templates with values), release management (install/upgrade/rollback with history), dependency management (sub-charts), and hooks for lifecycle events. It solves the problem of managing many YAML files across environments.

Q2. What is the difference between chart version and appVersion?

version (in Chart.yaml) is the chart's own semver version — incremented when chart templates or defaults change. appVersion is the version of the application being deployed (e.g., Docker image tag). Changing appVersion without changing version means the chart package itself is unchanged. Best practice: always bump chart version when releasing, keep appVersion aligned with app release.

Q3. How does --set differ from -f values file?

--set image.tag=v2.0 overrides a single value inline (dot-notation for nested). Multiple --set flags are allowed. Values set via --set override -f file values. -f prod-values.yaml loads an entire values file — cleaner for many overrides. Multiple -f files are merged left-to-right (later files win). Best practice: use -f for environment overrides committed to Git, --set for dynamic values (image tag in CI).

Q4. Explain the Helm hook execution order.

Hooks execute in order: pre-install -> install resources -> post-install. For upgrades: pre-upgrade -> upgrade resources -> post-upgrade. hook-weight controls ordering within the same hook type (lower weight runs first). Hooks are implemented as Jobs, Pods, or any k8s resource. Helm waits for Jobs to complete (success) before proceeding. hook-delete-policy: before-hook-creation deletes old Job before running new one.

Q5. What is helm rollback and what are its limitations?

helm rollback re-applies the manifests from the specified revision. It does NOT rollback: PVC data, database changes (run by migration hooks), ConfigMap-injected env vars that application read at startup (pod restart required). It DOES rollback: deployment image, replica count, service config, ingress rules. After rollback, helm history shows rollback as a new revision.

Q6. What is the difference between helm upgrade --install and helm install?

helm install fails if the release already exists. helm upgrade --install: installs if not exists, upgrades if exists — idempotent, CI-friendly. Combined with --atomic: if upgrade fails, automatically rollback to last good revision. --wait: block until all resources are Ready (Deployment pods, Services endpoints). Best practice in CI: helm upgrade --install --atomic --wait --timeout 300s.

Q7. What is Helmfile and when would you use it over plain Helm?

Helmfile is a declarative configuration for multiple Helm releases. Use when: 1) Multiple charts need to be deployed together (app + database + monitoring). 2) Multiple environments (dev/staging/prod) with different values. 3) Release ordering matters (needs: declares dependencies). 4) Team wants GitOps-style declarative state for all releases. Helmfile adds: diff (preview), sync (reconcile), destroy. Alternative: ArgoCD/Flux ApplicationSet for GitOps-native multi-app management.

Q8. How does Kustomize differ from Helm?

Helm: template engine (Go templates), packaging (chart.tgz), release management (history/rollback). Kustomize: YAML overlay engine, no templates/variables, pure patching. Kustomize is simpler (no learning curve for Go templates), built into kubectl. Helm is more powerful for complex parameterization and release lifecycle. Many teams use both: Helm for third-party charts, Kustomize for in-house app customization. ArgoCD supports both natively.

Q9. What is a Helm library chart?

A library chart (type: library in Chart.yaml) contains only named template partials (`{{ define 'mylib.labels' }}`) with no standalone templates. It cannot be installed directly. Other charts list it as a dependency and call its templates via `{{ include 'mylib.labels' . }}`. Use for: standardizing labels, resource names, security contexts across multiple charts. Reduces duplication in a chart monorepo.

Q10. How do you manage secrets in Helm charts?

Helm values are stored in k8s Secret (release data is base64 in a Secret named `sh.helm.release.v1.*`). Don't put plaintext passwords in values.yaml. Approaches: 1) helm-secrets plugin: uses SOPS to encrypt values files (AES256 + KMS). 2) Reference k8s Secrets in templates (secretKeyRef) — secret must exist separately. 3) External Secrets Operator creates secrets before Helm install (use pre-install hook check). 4) Vault Agent Injector annotation in chart templates.

Q11. What is the Helm values schema validation?

Helm supports values.schema.json (JSON Schema) in the chart root. Helm validates provided values against the schema before rendering. Catches type errors (string where number expected), missing required values, invalid enum values. Example: required: ['image', 'replicaCount'], image.tag must be string. Run: `helm lint` validates schema. Prevents misconfigured deploys in CI.

Q12. How does Helm dependency management work?

List dependencies in Chart.yaml under dependencies: with name, version (semver range), repository. helm dependency update: downloads dependency charts into charts/ directory and creates Chart.lock. Chart.lock: pinned exact versions (like package-lock.json). Commit Chart.lock to Git. During helm install, Helm uses charts/ directory (don't commit .tgz files, they're large). condition: field allows enabling/disabling sub-charts via values.

Q13. What is helm test and how do you write test cases?

helm test : runs Pods annotated with helm.sh/hook: test. Test Pod runs a command (curl, psql, test script) and must exit 0 for success. Example: `helm test my-app` runs a Pod that curls the health endpoint and checks HTTP 200. Tests run in a Job-like fashion post-install. Integrate with: `helm upgrade --install && helm test`. Failed helm test doesn't rollback automatically — combine with CI assertions.

Q14. What is the difference between helm diff and helm template?

helm template: renders templates locally to YAML (no k8s connection needed). Good for: reviewing what will be applied, passing to kubectl apply, CI validation. helm diff (plugin): compares current cluster state with new release (kubectl diff equivalent). Shows exactly what will change before helm upgrade. Essential for reviewing infrastructure changes in PR review. Install: `helm plugin install https://github.com/databus23/helm-diff`.

Q15. How do you handle chart versioning in a monorepo?

Use chart-releaser (cr) tool or GitHub Actions helm/chart-releaser-action. On merge to main: `cr lint+install` validates changes, chart-releaser packages and uploads changed charts to GitHub Releases, updates gh-pages branch as chart repository. Semantic versioning: bump patch for bug fixes, minor for new features, major for breaking changes. Always increment chart version even if only appVersion changes.

Q16. What is ArgoCD's relationship to Helm?

ArgoCD supports Helm as an application source type. ArgoCD renders Helm templates server-side using helm template with provided values. It then manages the rendered manifests as an Application. ArgoCD shows diff between desired (rendered) and live (cluster) state. It does NOT use helm install/upgrade — it uses kubectl apply (no helm history in cluster). Helm hooks are not executed by ArgoCD (use ArgoCD Sync Waves instead for ordering).

Q17. How do you debug a failed Helm install?

1) `helm install --debug --dry-run`: validate templates and values before applying. 2) `helm template >/tmp/rendered.yaml && kubectl apply --dry-run=server -f /tmp/rendered.yaml`. 3) After failed install: `helm list -a` (shows failed releases), `kubectl get events -n ns`. 4) `helm get manifest`: shows last applied manifests. 5) `helm get values`: shows values used for release. 6) `kubectl describe pod/deployment` for specific errors.

Q18. What are the security considerations for Helm?

1) Helm stores release metadata (including values!) as Secrets in the cluster — protect with RBAC. 2) Never put passwords in values.yaml committed to Git. 3) Validate chart provenance: `helm install --verify` (requires chart signing with `helm pgp`). 4) Restrict helm RBAC to namespaced service accounts (avoid cluster-admin for helm SA). 5) Use OCI registries with image signing (Cosign) for chart supply chain security. 6) Audit third-party charts before using (review templates for RBAC, hostPath, privileged containers).

Q19. What is the difference between `helm upgrade --force` and `--cleanup-on-fail`?

`--force`: for immutable fields, deletes and recreates resources that cannot be patched (e.g., changing Service type from ClusterIP to NodePort). Causes brief downtime. `--cleanup-on-fail`: if upgrade fails, delete newly created resources (prevents partial state). Combined with `--atomic`: auto-rollback on failure + cleanup of failed resources. Best practice for production: `--atomic --cleanup-on-fail --wait --timeout 600s`.

Q20. How does helm handle CRD upgrades?

Helm installs CRDs from `crds/` directory on first install but does NOT upgrade CRDs on helm upgrade. This is intentional — CRD schema changes can be destructive. To upgrade CRDs: use a pre-upgrade hook Job (`kubectl apply --server-side -f crd.yaml`), or use separate CRD chart, or use kubectl manually. Note: `helm uninstall` does NOT delete CRDs by default (protect against data loss). Use `--no-hooks` to skip hook execution during uninstall.

Q21. What is the Helmfile environments feature?

environments: in `helmfile.yaml` defines named environments (dev, staging, prod) each with their own values files and secret files (`helm-secrets`). `helmfile -e prod sync` applies prod environment config. Within `helmfile.yaml`, `{{ .Environment.Name }}` accesses current env name. `requiredEnv 'IMAGE_TAG'` fails if env var not set. environments enable a single `helmfile.yaml` to manage all environments consistently.

Q22. How do you implement blue-green deployments with Helm?

1) Maintain two releases: `my-app-blue` and `my-app-green` with different selectors. 2) Service selector points to active color: `app: my-app, color: blue`. 3) Deploy new version to inactive color: `helm upgrade my-app-green --set color=green`. 4) Test green deployment. 5) Switch service selector: `helm upgrade my-app-service --set activeColor=green`. 6) Keep blue running for quick rollback: update service selector back. Argo Rollouts provides a more sophisticated blue-green implementation.

Q23. What is the `Kustomize patches` vs `patchesStrategicMerge` difference?

`patchesStrategicMerge` (deprecated): list of YAML files containing partial resource definitions merged using Kubernetes strategic merge patch rules (lists merged by name). `patches` (current): supports both strategic merge patches (for k8s resources) and JSON 6902 patches (precise path-based operations: add, replace, remove). JSON patches: more explicit, better for complex changes. Strategic merge: more natural for adding/updating spec fields.

Q24. How do you pass dynamic values to Helm in CI/CD?

Use `--set` for runtime values: `--set image.tag=${GIT_SHA} --set deployment.timestamp=$(date +%s)`. Environment-specific values: `-f environments/${ENV}/values.yaml` committed to Git. Secrets: use `helm-secrets` plugin (SOPS encrypted values files) or reference k8s Secrets. Image tag strategy: use Git SHA

for immutable tags, never 'latest' in production. Example: `helm upgrade --install app ./chart -f prod.yaml --set image.tag=${CI_COMMIT_SHA}`.

Q25. What happens to Helm history on cluster recreate?

Helm stores release history as Secrets (`sh.helm.release.v1..v`) in the release namespace. If cluster is destroyed and recreated, Helm history is lost. Reinstalling a chart creates revision 1 again. Best practice: store chart + values in Git (source of truth). Velero can backup/restore Helm release Secrets as part of namespace backup. `historyMax` in Helm limits history retention (default 10 revisions per release).

Q26. What is helm show and when do you use it?

`helm show chart` : shows Chart.yaml metadata (version, appVersion, description, dependencies). `helm show values` : shows default values.yaml — essential before installing to understand options. `helm show readme` : shows README.md documentation. `helm show all` : all of the above. Works with local charts, repository charts, and OCI charts. `helm search repo` finds available charts in added repositories.